

求解方案设计

滚动时域混合整数规划方法 RHC-MILP

1. 方法概述

针对热-电协同约束下的星载多核任务调度问题，可以设计一种基于滚动时域的混合整数线性规划方法，即 Rolling Horizon Mixed-Integer Linear Programming，简称 RHC-MILP。

该方法的核心思想是：在每一个离散决策时刻 t_0 ，调度器并不只根据当前时隙的系统状态做局部决策，而是利用未来 H 个时隙内的太阳能输入、基础功耗、热环境输入和任务队列信息，构建一个有限预测窗口内的混合整数优化问题。优化器在该窗口内同时决定任务块的启动时间、运行状态和核心分配，并在满足多核容量约束、任务顺序约束、能量约束和热约束的条件下，最小化任务完成时延、逾期惩罚和窗口末端剩余工作量惩罚。

与一次性求解全局调度问题不同，RHC-MILP 采用滚动优化思想。每次只执行当前时刻 t_0 的调度结果，然后根据实际执行情况更新电池电量、芯片温度和任务队列状态，再滚动到下一时隙 $t_0 + 1$ 重新建模和求解。该机制能够适应卫星轨道环境中太阳能输入周期变化、任务动态到达以及热-电状态持续演化等特征。

2. 方法适用性分析

本文问题具有以下特点：

- 星载计算平台为多核处理器，不同核心在执行不同任务块时可能具有不同执行长度和功耗；
- 任务由多个顺序执行的原子任务块构成，任务块不能在任意位置抢占，只能在自然边界处切换；
- 系统能量状态具有跨时隙耦合性，当前调度决策会影响未来电池电量；
- 系统热状态同样具有跨时隙累积效应，当前高功耗任务可能导致未来热约束违约；
- 任务具有释放时间、软截止期和重要性权重；
- 调度目标不仅关注当前时隙收益，还需要避免将大量任务推迟到未来窗口之外。

因此，该问题本质上是一个带动态状态约束的多阶段混合整数优化问题。RHC-MILP 能够显式描述任务启动、任务运行、多核分配、热电状态演化和任务时延目标，因此适合作为本文的求解方法。

3. 基本符号定义

3.1 集合与索引

符号	含义
----	----

$J = \{1, 2, \dots, N\}$	任务集合
$K = \{1, 2, \dots, C\}$	处理器核心集合
$M_j = \{1, 2, \dots, M_j\}$	任务 j 的任务块集合
t	离散时间槽索引
t_0	当前滚动决策时刻
H	预测窗口长度
$H(t_0) = \{t_0, t_0 + 1, \dots, t_0 + H - 1\}$	当前预测窗口
Δt	单个时隙长度

3.2 任务参数

符号	含义
r_j	任务 j 的释放时刻
d_j	任务 j 的软截止期或期望完成时刻
w_j	任务 j 的重要性权重
$\ell_{j,m,c}$	任务 j 的第 m 个任务块在核心 c 上的执行长度
$p_{j,m,c}$	任务 j 的第 m 个任务块在核心 c 上执行时的平均动态功率

3.3 系统参数

符号	含义
$E(t)$	时刻 t 的电池剩余能量
$E_{\max}$	电池最大容量
$E_{\min}$	电池最低安全能量

$P_{\text{harv}}(t)$	时刻 t 的太阳能采集功率
$P_{\text{base}}(t)$	时刻 t 的平台基础功耗
$P_{\text{com}}(t)$	时刻 t 的计算载荷功耗
$T(t)$	时刻 t 的芯片或载荷温度
T_{max}	系统允许的最高安全温度
C_{th}	热容量
$B(\tau)$	当前窗口内从 t_0 到 τ 的可用热预算

4. 滚动时域建模思想

在当前时刻 t_0 ，调度器观测系统状态：

$$E(t_0 - 1), \quad T(t_0 - 1), \quad Que(t_0)$$

其中， $Que(t_0)$ 表示当前时刻已经释放且尚未完成的任务队列。

调度器进一步获得未来窗口内的预测信息：

$$P_{\text{harv}}(\tau), \quad P_{\text{base}}(\tau), \quad B(\tau), \quad \tau \in H(t_0)$$

然后在窗口 $H(t_0)$ 内求解一个混合整数规划问题。虽然优化器会规划整个窗口内的任务执行方案，但实际只执行当前时隙 t_0 的调度结果。下一时隙到来后，系统重新观测状态并再次优化。

该过程可表示为：

$$\text{Observe} \rightarrow \text{Predict} \rightarrow \text{Optimize} \rightarrow \text{Execute First Slot} \rightarrow \text{Update} \rightarrow \text{Roll Forward}$$

5. 决策变量设计

5.1 任务块启动变量

定义二元变量：

$$s_{j,m,t,c} \in \{0,1\}$$

其中：

$$s_{j,m,t,c} = 1$$

表示任务 j 的第 m 个任务块在时刻 t 于核心 c 上启动执行；

$$s_{j,m,t,c} = 0$$

表示该任务块不在该时刻和该核心上启动。

5.2 任务块运行变量

定义二元变量：

$$x_{j,m,\tau,c} \in \{0, 1\}$$

其中：

$$x_{j,m,\tau,c} = 1$$

表示任务 j 的第 m 个任务块在时刻 τ 正在核心 c 上执行；

$$x_{j,m,\tau,c} = 0$$

表示该任务块在该时刻未在该核心上执行。

5.3 任务完成时间变量

定义连续变量：

$$C_j = \sum_{c \in K} \sum_t (t + \ell_{j,m,c} - 1) s_{j,m,c,t}, \quad \forall j, \forall m$$

表示任务 j 的整体完成时刻。

由于任务 j 由多个顺序任务块构成，只有当其所有任务块均完成后，任务 j 才算完成。因此， C_j 应等于任务 j 所有任务块完成时刻中的最大值。

5.4 拖期变量

定义连续变量：

$$L_j \geq 0$$

表示任务 j 相对于软截止期 d_j 的拖期时间：

$$L_j = \max(0, C_j - d_j)$$

5.5 剩余工作量变量

为避免滚动窗口只关注当前时隙而忽略窗口之后的任务积压，引入窗口末端剩余工作量惩罚项。

定义：

$$R_j(t_0 + H)$$

表示任务 j 在当前预测窗口结束时尚未完成的剩余工作量。

6. 辅助变量与中间量

6.1 计算功率

在时隙 τ ，系统总计算功率为：

$$P_{\text{com}}(\tau) = \sum_{c \in K} \sum_j \sum_m p_{j,m,c} x_{j,m,\tau,c}$$

该式表示所有核心上正在运行任务块的动态功率总和。

6.2 负载消耗能量

时隙 τ 的总负载消耗能量为：

$$E_{\text{load}}(\tau) = (P_{\text{base}}(\tau) + P_{\text{com}}(\tau)) \Delta t$$

其中， $P_{\text{base}}(\tau)$ 是平台基础功耗， $P_{\text{com}}(\tau)$ 是计算载荷功耗。

6.3 太阳能输入能量

时隙 τ 的太阳能输入能量为：

$$E_{\text{total}}(\tau) = P_{\text{harv}}(\tau) \Delta t$$

6.4 累计计算热量

从当前时刻 t_0 到未来时刻 τ 的累计计算热量为：

$$Q_{\text{cum}}(\tau) = \sum_{u=t_0}^{\tau} Q(u)$$

其中，计算功耗被近似转化为等量计算产热。

7. 目标函数设计

本文采用“任务响应时间 + 拖期惩罚 + 窗口末端剩余工作量惩罚”的综合目标函数：

$$\min J(t_0) = \alpha \sum_j w_j (C_j - r_j) + \beta \sum_j w_j L_j + \mu \sum_j w_j R_j(t_0 + H)$$

其中：

- α 表示任务完成时延项权重；
- β 表示拖期惩罚项权重；
- μ 表示剩余工作量惩罚项权重；
- w_j 表示任务 j 的重要性权重。

7.1 完成时延项

$$\sum_{j \in J} w_j (C_j - r_j)$$

该项用于衡量任务从释放到完成经历的总响应时间。该项越小，说明任务平均完成时延越低。

7.2 拖期惩罚项

$$\sum_{j \in J} w_j L_j$$

该项用于惩罚任务超过软截止期 d_j 的情况。任务越重要，其权重 w_j 越大，逾期带来的惩罚越高。

7.3 剩余工作量惩罚项

$$\sum_{j \in J} w_j R_j(t_0 + H)$$

该项用于避免滚动优化中的短视行为。如果不加入该项，优化器可能只安排窗口前部容易完成或低功耗任务，而将大量高功耗任务推迟到窗口之后，导致未来调度压力过大。

8. 约束条件设计

8.1 启动变量与运行变量关系

如果任务块 (j, m) 在时刻 t 于核心 c 上启动，且其执行长度为 $\ell_{j, m, c}$ ，则该任务块会占用从 t 到 $t + \ell_{j, m, c} - 1$ 的连续时隙。

因此，运行变量与启动变量之间满足：

$$x_{j, m, \tau, c} = \sum_{t=\max(t_0, \tau-\ell_{j, m, c}+1)}^{\tau} s_{j, m, t, c}$$

其中：

$$\tau \in H(t_0)$$

该约束表示：只要任务块在能够覆盖时隙 τ 的某个时刻启动，那么它在时隙 τ 就处于运行状态。

8.2 每个任务块至多启动一次约束

在滚动窗口内，每个任务块最多只能被启动一次：

$$\sum_{c \in K} \sum_{t=t_0}^{t_0+H-\ell_{j, m, c}} s_{j, m, t, c} \leq 1$$

该约束适用于滚动窗口优化。原因是有些任务块可能由于能量或热约束限制，在当前窗口内无法安全执行，因此不能强制其必须在当前窗口内启动。

如果进行离线全局调度，且所有任务必须在全局调度周期内完成，则可改为：

$$\sum_{c \in K} \sum_{t=r_j}^{T-\ell_{j, m, c}+1} s_{j, m, t, c} = 1, \quad \forall j, m$$

8.3 释放时间约束

任务 j 在释放时刻 r_j 之前不能被调度。因此：

$$s_{j, m, t, c} = 0, \quad \forall t < r_j$$

在滚动窗口内，如果 $r_j > t$ ，则该任务块不能在时刻 t 启动。

8.4 任务内部顺序约束

任务 j 内部的任务块必须按照顺序执行。即第 $m+1$ 个任务块必须在第 m 个任务块完成后才能开始。

首先定义任务块 (j,m) 的启动时刻：

$$S_{j,m} = \sum_{c \in K} \sum_{t \in H(t_0)} t s_{j,m,t,c}$$

定义任务块 (j,m) 的完成时刻：

$$F_{j,m} = \sum_{c \in K} \sum_{t \in H(t_0)} (t + \ell_{j,m,c} - 1) s_{j,m,t,c}$$

则任务顺序约束为：

$$S_{j,m+1} \geq F_{j,m} + 1, \quad \forall j, \quad m = 1, 2, \dots, M_j - 1$$

但由于任务块可能被分配到不同核心，实际建模时更推荐使用基于启动变量求和的形式。

在滚动窗口中，如果任务块 (j,m) 已经在历史时隙完成，则任务块 $(j,m+1)$ 可以直接进入就绪集合；如果任务块 (j,m) 尚未完成，则任务块 $(j,m+1)$ 不能在当前窗口中启动。

8.5 多核容量约束

每个处理器核心在每个时隙最多只能执行一个任务块：

$$\sum_{j \in J} \sum_{m \in M_j} x_{j,m,\tau,c} \leq 1, \quad \forall c \in K, \quad \forall \tau \in H(t_0)$$

该约束保证同一核心不会在同一时刻并行执行多个任务块。

8.6 非抢占执行约束

由于任务块是按照 tile、strip、granule 或 burst 等自然边界划分的原子任务块，因此任务块一旦启动，就必须连续执行直到该任务块完成。

非抢占性已经通过启动变量与运行变量关系隐式保证。也就是说，如果 $s_{j,m,t,c} = 1$ ，则：

$$x_{j,m,t,c} = 1, \quad x_{j,m,t+1,c} = 1, \quad \dots, \quad x_{j,m,t+\ell_{j,m,c}-1,c} = 1$$

因此不需要额外引入中断变量。

8.7 任务完成时间约束

任务 j 的整体完成时间 C_j 应不早于该任务所有任务块的完成时刻。因此可使用 Big-M 线性化约束：

$$C_j \geq t + \ell_{j,m,c} - 1 - M^{\text{big}} (1 - s_{j,m,t,c}), \quad \forall j, \forall m, \forall t, \forall c$$

其中， M^{big} 是 Big-M 方法中的足够大常数。

当 $s_{j,m,t,c} = 1$ 时，约束变为：

$$C_j \geq t + \ell_{j,m,c} - 1$$

表示任务 (j) 的完成时间不能早于该任务块的完成时间。

当 $s_{j,m,t,c} = 0$ 时，约束变为：

$$C_j \geq t + \ell_{j,m,c} - 1 - M^{\text{big}}$$

若 M^{big} 足够大，则该约束被放松，不会对 C_j 产生实际限制。

在滚动窗口内，可以取：

$$M^{\text{big}} = H + \max_{j,m,c} \ell_{j,m,c}$$

在全局调度模型中，可以取：

$$M^{\text{big}} = T + \max_{j,m,c} \ell_{j,m,c}$$

需要注意， M^{big} 不能过小，否则可能错误排除可行解；也不宜过大，否则会导致MILP求解器数值稳定性下降。

8.8 拖期变量约束

拖期变量 L_j 用于表示任务完成时间超过软截止期的部分：

$$L_j \geq C_j - d_j, \quad \forall j$$

$$L_j \geq 0, \quad \forall j$$

由于目标函数中最小化 L_j ，因此最终有：

$$L_j = \max(0, C_j - d_j)$$

8.9 计算功率约束

时隙 τ 的计算功率由所有正在运行的任务块功率累加得到：

$$P_{\text{com}}(\tau) = \sum_{c \in K} \sum_{j \in J} \sum_{m \in M_j} p_{j,m,c} x_{j,m,\tau,c}, \quad \forall \tau \in H(t_0)$$

该约束将调度决策变量 $x_{j,m,\tau,c}$ 与系统功耗状态连接起来。

8.10 能量状态更新约束

在每个时隙中，电池能量由上一时隙电量、太阳能输入和系统负载消耗共同决定。

为了便于线性建模，可采用如下能量递推形式：

$$E(\tau) = E(\tau - 1) + P_{\text{harv}}(\tau)\Delta t - (P_{\text{base}}(\tau) + P_{\text{com}}(\tau))\Delta t, \quad \forall \tau \in H(t_0)$$

其中：

$$E(t_0 - 1)$$

为当前滚动窗口开始前的已知电池状态。

8.11 电池容量约束

电池能量不能超过最大容量，也不能低于最低安全电量：

$$E_{\min} \leq E(\tau) \leq E_{\max}, \quad \forall \tau \in H(t_0)$$

其中：

$$E_{\min} = (1 - DoD)E_{\max}$$

DoD 表示电池允许的最大放电深度。

该约束可避免由于过度放电导致电池安全风险。

8.12 能量可行性约束的等价形式

如果希望与原始能量可行性判据保持一致，也可以显式定义：

$$E_{\text{load}}(\tau) = (P_{\text{base}}(\tau) + P_{\text{com}}(\tau)) \Delta t$$

$$E_{\text{total}}(\tau) = P_{\text{harv}}(\tau) \Delta t$$

$$E_{\text{deficit}}(\tau) \geq E_{\text{load}}(\tau) - E_{\text{total}}(\tau)$$

$$E_{\text{deficit}}(\tau) \geq 0$$

$$E_{\text{avail}}(\tau) = E(\tau - 1) - E_{\min}$$

能量可行性要求：

$$E_{\text{deficit}}(\tau) \leq E_{\text{avail}}(\tau), \quad \forall \tau \in H(t_0)$$

该形式与“若能量缺口大于安全可放电量，则时刻 τ 出现能量不可行”的判据一致。

8.13 热预算约束

由于星载平台散热能力有限，任务调度必须保证计算产热不会使系统超过安全温度上限。

在滚动窗口内，将热约束转化为累计热量预算约束：

$$Q_{\text{cum}}(\tau) = \sum_{u=t_0}^{\tau} P_{\text{com}}(u) \Delta t, \quad \forall \tau \in H(t_0)$$

定义从当前温度 $T(t_0 - 1)$ 出发，到未来时刻 τ 的可用热预算为

$$B(\tau) = \sum_{u=t_0}^{\tau} Q^{(e)}(u) + C_{\text{th}} (T_{\max} - T(t_0 - 1))$$

其中， $Q^{(e)}(u)$ 表示时隙 u 的等效散热余量或热环境修正项， $C_{\text{th}} (T_{\max} - T(t_0 - 1))$ 表示当前温度距离安全温度上限所对应的热容空间。

热可行性约束为：

$$Q_{\text{cum}}(\tau) \leq B(\tau), \quad \forall \tau \in H(t_0)$$

该约束表示，从当前时刻到任意未来时刻 τ 的累计计算产热都不能超过系统在该时段内可承受的热预算。

8.14 剩余工作量惩罚建模

为了定义窗口末端剩余工作量，引入变量：

$$y_{j,m,c}^H \in \{0, 1\}$$

其中：

$$y_{j,m,c}^H = 1$$

表示任务块 (j,m) 在当前预测窗口内被启动并完成；

$$y_{j,m,c}^H = 0$$

表示该任务块未能在当前预测窗口内完成。

可定义：

$$y_{j,m,c}^H = \sum_{\tau=t_0}^{t_0+H-\ell_{j,m,c}} s_{j,m,\tau,c}$$

则任务 j 的窗口末端剩余工作量为：

$$R_j(t_0 + H) = \sum_{m \in M_j} \sum_{c \in K} \ell_{j,m,c} (1 - y_{j,m,c}^H)$$

在实际实现中，为避免同一任务块在多个核心上的执行长度重复计入，可以进一步定义与核心无关的完成指示变量：

$$y_{j,m}^H = \sum_{c \in K} \sum_{\tau=t_0}^{t_0+H-\ell_{j,m,c}} s_{j,m,\tau,c}$$

并将剩余工作量写为：

$$R_j(t_0 + H) = \sum_{m \in M_j} \bar{\ell}_{j,m} (1 - y_{j,m}^H)$$

其中， $\bar{\ell}_{j,m}$ 可取任务块在不同核心上的平均执行长度、最短执行长度或根据预估核心分配得到的执行长度。

9. 完整 RHC-MILP 模型

综上，在当前滚动时刻 t_0 ，RHC-MILP 可表示为：

$$\min \quad \alpha \sum_{j \in J} w_j (C_j - r_j) + \beta \sum_{j \in J} w_j L_j + \mu \sum_{j \in J} w_j R_j(t_0 + H)$$

满足：

$$x_{j,m,\tau,c} = \sum_{t=\max(t_0, \tau-\ell_{j,m,c}+1)}^{\tau} s_{j,m,t,c}$$

$$\sum_{c \in K} \sum_{t=t_0}^{t_0+H-\ell_{j,m,c}} s_{j,m,t,c} \leq 1$$

$$s_{j,m,t,c} = 0, \quad t < r_j$$

$$S_{j,m+1} \geq F_{j,m} + 1$$

$$\sum_{j \in J} \sum_{m \in M_j} x_{j,m,\tau,c} \leq 1$$

$$C_j \geq t + \ell_{j,m,c} - 1 - M^{\text{big}} (1 - s_{j,m,t,c})$$

$$L_j \geq C_j - d_j$$

$$L_j \geq 0$$

$$P_{\text{com}}(\tau) = \sum_{c \in K} \sum_{j \in J} \sum_{m \in M_j} p_{j,m,c} x_{j,m,\tau,c}$$

$$E(\tau) = E(\tau - 1) + P_{\text{harv}}(\tau) \Delta t - (P_{\text{base}}(\tau) + P_{\text{com}}(\tau)) \Delta t$$

$$E_{\min} \leq E(\tau) \leq E_{\max}$$

$$Q_{\text{cum}}(\tau) = \sum_{u=t_0}^{\tau} P_{\text{com}}(u) \Delta t$$

$$Q_{\text{cum}}(\tau) \leq B(\tau)$$

$$s_{j,m,t,c} \in \{0, 1\}$$

$$x_{j,m,\tau,c} \in \{0, 1\}$$

$$C_j \geq 0, \quad L_j \geq 0$$

10. 滚动求解流程

RHC-MILP 的完整求解流程如下。

Step 1: 观测当前系统状态

在每个决策时刻 t_0 ，调度器读取：

$$E(t_0 - 1), \quad T(t_0 - 1), \quad Que(t_0)$$

其中， $Que(t_0)$ 包括已经释放但尚未完成的任务，以及其当前可执行任务块。

Step 2: 生成预测窗口

设定预测窗口：

$$H(t_0) = \{t_0, t_0 + 1, \dots, t_0 + H - 1\}$$

在该窗口内预测：

$$Pharv(\tau), \quad Pbase(\tau), \quad B(\tau)$$

Step 3：构造就绪任务块集合

根据任务释放时间和前驱任务块完成情况，构造当前可调度任务块集合：

$$R(t_0) = \{(j, m) \mid r_j \leq t_0, (j, m - 1) \text{ 已完成或 } m = 1\}$$

只有属于就绪集合的任务块才允许被调度。

Step 4：建立 MILP 模型

根据当前窗口内的任务集合、核心集合、功耗表、执行长度表、能量预测和热预算，建立 RHC-MILP 模型。

Step 5：调用求解器求解

可使用如下 MILP 求解器：

- Gurobi;
- CPLEX;
- SCIP;
- HiGHS;
- CBC;

对于实时性要求较高的场景，可以设置最大求解时间：

$$T_{\text{solve}}^{\max}$$

若在规定时间内未达到最优解，可接受当前最优可行解。

Step 6：执行第一时隙调度结果

虽然优化器得到的是整个窗口内的调度计划，但实际只执行当前时隙 t_0 的任务分配：

$$x_{j,m,t_0,c}$$

即决定每个核心在当前时隙执行哪个任务块，或保持空闲。

Step 7：更新系统状态

根据实际执行结果更新：

$$E(t_0), T(t_0), Que(t_0 + 1)$$

同时记录已完成任务块，并释放其后继任务块。

Step 8：窗口滚动

令：

$$t_0 \leftarrow t_0 + 1$$

然后重复 Step 1 至 Step 7，直到所有任务完成或仿真周期结束。

11. 算法伪代码

代码块

```
1  Algorithm: RHC-MILP for Thermo-Electric-Aware Onboard Task Scheduling
2
3  Input:
4      Task set J
5      Core set K
6      Current time  $t_0$ 
7      Prediction horizon H
8      Battery state  $E(t_0-1)$ 
9      Temperature state  $T(t_0-1)$ 
10     Ready queue  $Que(t_0)$ 
11     Predicted solar power  $P_{harv}(t)$ ,  $t$  in  $[t_0, t_0+H-1]$ 
12     Base power  $P_{base}(t)$ ,  $t$  in  $[t_0, t_0+H-1]$ 
13     Thermal budget  $B(t)$ ,  $t$  in  $[t_0, t_0+H-1]$ 
14
15  Output:
16     Current-slot scheduling decision  $\{x_{\{j,m,t_0,c\}}\}$ 
17
18  1: while unfinished tasks exist do
19  2:     Observe current system state  $E(t_0-1)$ ,  $T(t_0-1)$ ,  $Que(t_0)$ 
20  3:     Construct prediction window  $H(t_0) = \{t_0, \dots, t_0+H-1\}$ 
21  4:     Predict solar input, base power, and thermal budget in  $H(t_0)$ 
22  5:     Build ready block set  $R(t_0)$ 
23  6:     Formulate RHC-MILP model:
24  7:         Minimize weighted response time, tardiness, and remaining workload
25  8:         Subject to:
26  9:             start-run relationship constraints
27 10:             block execution constraints
28 11:             precedence constraints
29 12:             multicore capacity constraints
30 13:             completion time constraints
31 14:             battery energy constraints
32 15:             thermal budget constraints
33 16:     Solve RHC-MILP using MILP solver
34 17:     if feasible solution exists then
35 18:         Execute only the first-slot decision  $\{x_{\{j,m,t_0,c\}}\}$ 
36 19:     else
37 20:         Execute safe fallback policy
38 21:     end if
```

```
39 22:    Update  $E(t_0)$ ,  $T(t_0)$ ,  $Que(t_0+1)$ 
40 23:     $t_0 = t_0 + 1$ 
41 24: end while
```